Filière : Transformation Digital Industrielle

Module : Génie Logiciel

TD. Docker

Application Multi-Conteneurs

Réalisée par : Nawal El Aourfe

Encadrant : Pr. BE.ELBAGHAZAOUI

ENSA Beni Mellal — Année universitaire 2025/2026

Étape 1 : Préparer l'application

Structure du projet créée :

La première étape consiste à créer l'arborescence du projet

```
C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker>mkdir project

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker>cd project

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>mkdir frontend

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>mkdir backend
```

house - Copie > tp docker > project

Nom	Modifié le	Type	Taille
backend	20/04/2026 16:05	Dossier de fichiers	
frontend	20/04/2026 16:05	Dossier de fichiers	
docker-compose.yml	20/04/2026 21:22	Fichier source Yaml	1 Ko

Étape 2 : Créer un Dockerfile

Création du Dockerfile pour le service frontend basé sur Nginx Alpine :

```
C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo FROM nginx:alpine > frontend\Dockerfile

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo COPY index.html /usr/share/nginx/html/index.html >> frontend\Dockerfile

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo EXPOSE 80 >> frontend\Dockerfile

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo CMD ["nginx", "-g", "daemon off;"] >> frontend\Dockerfile

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>
```

Commandes exécutées dans CMD :

Création de la structure de répertoires et du fichier index.html :

```
C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo *.log > frontend\.dockerignore
```

Interprétation :

FROM nginx:alpine : On part d'une image Nginx légère basée sur Alpine Linux, ce qui réduit considérablement la taille de l'image finale.

COPY index.html /usr/share/nginx/html/index.html : On place le fichier HTML dans le répertoire où Nginx cherche les fichiers à afficher. **EXPOSE 80** : On indique que le conteneur écoute sur le port 80 (port HTTP standard).

CMD ["nginx", "-g", "daemon off;"] : On définit la commande pour lancer Nginx en avant-plan au démarrage du conteneur. Le fichier **.dockerignore** avec ***.log** évite d'inclure des fichiers journaux inutiles dans l'image, optimisant ainsi le processus de build.

Étape 3: configurer la base de donnees

```
C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo POSTGRES_USER=admin > backend\db.env

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo POSTGRES_PASSWORD=adminpassword >> backend\db.env

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo POSTGRES_DB=myapp >> backend\db.env
```

Interprétation : Le fichier **db.env** centralise les variables d'environnement sensibles de la base de données.
POSTGRES_USER=admin définit le nom d'utilisateur,

POSTGRES_PASSWORD=adminpassword le mot de passe, et

POSTGRES_DB=myapp le nom de la base de données à créer au démarrage du conteneur PostgreSQL.
Externaliser ces informations dans un fichier séparé respecte les bonnes pratiques de sécurité : les secrets ne sont pas codés en dur dans le **docker-compose.yml** ni dans les images Docker.

Étape 4 : créer un fichier docker-compose.yml

Contenu du Docker-compose :

```
C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo version: "3.8" > docker-compose.yml

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo services: >> docker-compose.yml

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo   frontend: >> docker-compose.yml

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo     build: ./frontend >> docker-compose.yml

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo     ports: >> docker-compose.yml

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo       - "8080:80" >> docker-compose.yml

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo     networks: >> docker-compose.yml

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo       - app-network >> docker-compose.yml

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo   db: >> docker-compose.yml

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo     image: postgres:13 >> docker-compose.yml

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo     environment: >> docker-compose.yml

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo       - POSTGRES_USER=${POSTGRES_USER} >> docker-compose.yml

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo       - POSTGRES_PASSWORD=${POSTGRES_PASSWORD} >> docker-compose.yml

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo       - POSTGRES_DB=${POSTGRES_DB} >> docker-compose.yml

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo     env_file: >> docker-compose.yml

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>echo       - ./backend/db.env >> docker-compose.yml

C:\Users\hp\Desktop\smart_warehouse - Copie\tp docker\project>(
Plus ? echo     volumes:
Plus ? echo       - db-data:/var/lib/postgresql/data
Plus ? echo     networks:
Plus ? echo       - app-network
Plus ? echo   pgadmin:
Plus ? echo     image: dpage/pgadmin4
Plus ? echo     environment:
Plus ? echo       - PGADMIN_DEFAULT_EMAIL=admin@admin.com
Plus ? echo       - PGADMIN_DEFAULT_PASSWORD=admin
Plus ? echo     ports:
Plus ? echo       - "5050:80"
Plus ? echo     depends_on:
Plus ? echo       - db
Plus ? echo     networks:
Plus ? echo       - app-network
Plus ? echo volumes:
Plus ? echo   db-data:
Plus ? echo networks:
```

Interprétation : Le fichier **docker-compose.yml** orchestre trois services interconnectés via le réseau **app-network** : • **Service frontend** : construit l'image depuis le Dockerfile local et expose le port 8080 de l'hôte vers le port 80 du conteneur. • **Service db** : utilise l'image officielle **postgres:13** et charge les variables depuis db.env. Le volume **db-data** assure la persistance des données même après redémarrage du conteneur. • **Service pgAdmin** : interface web d'administration PostgreSQL, accessible sur le port 5050. La directive **depends_on: db** garantit que la base de données démarre avant pgAdmin.

Étape 5 : Tester et valider

5.1 — Construction et démarrage des conteneurs

Exécution de la commande docker-compose up --build :

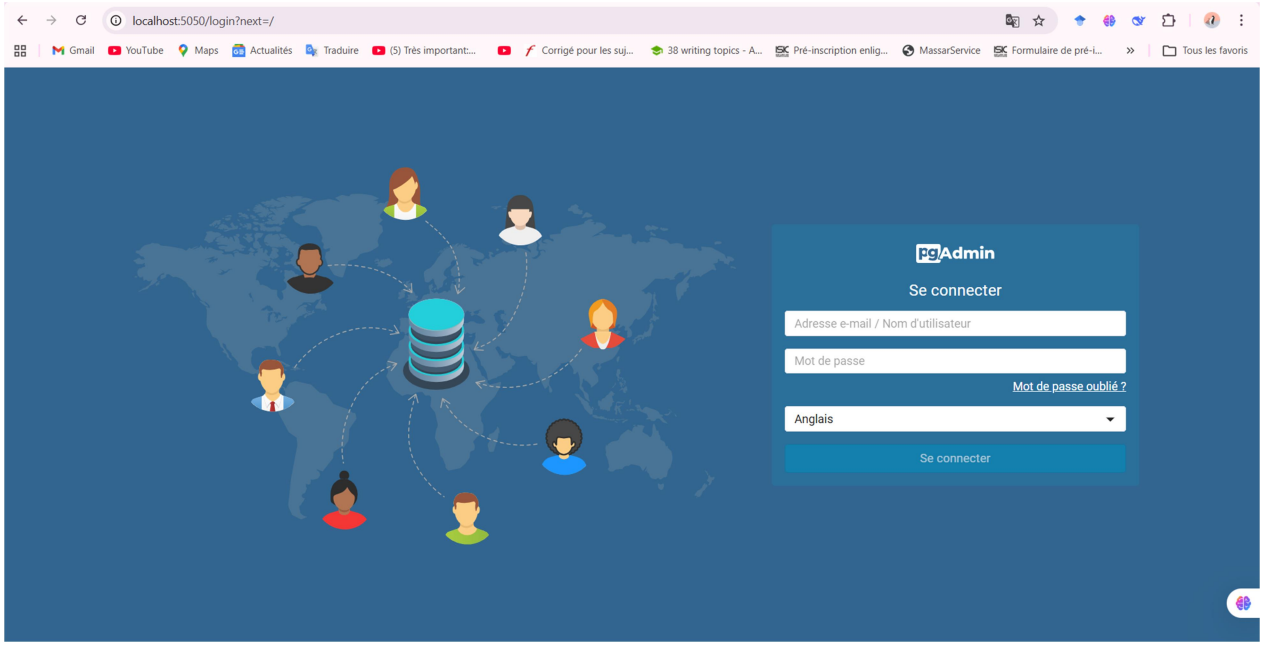
Définition des trois services : frontend (Nginx), db (PostgreSQL) et pgAdmin :

5.2 — Vérification du frontend sur http://localhost:8080

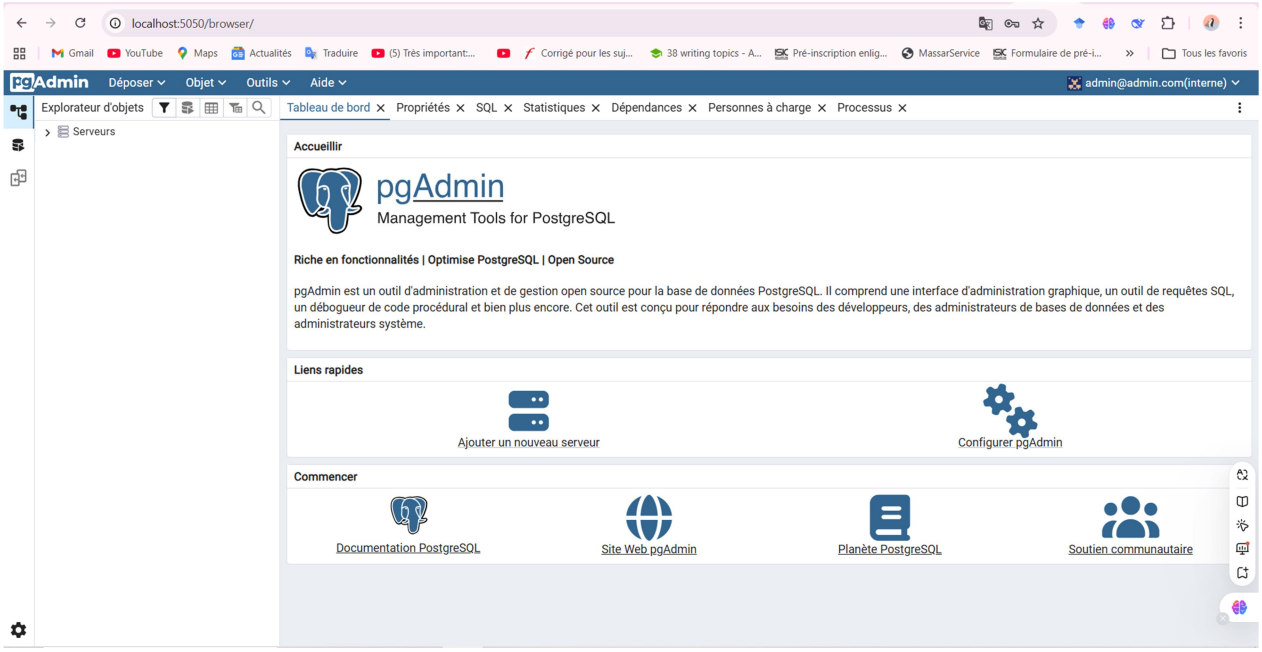
Interprétation : La commande **docker-compose up --build** effectue deux actions simultanément : elle reconstruit les images (flag **--build**) en exécutant les instructions du Dockerfile, puis démarre tous les services définis. On observe dans les logs que Docker télécharge les images manquantes (postgres:13, dpage/pgadmin4), construit l'image Nginx personnalisée, puis initialise les trois conteneurs dans l'ordre défini par les dépendances.

Interprétation : Le navigateur affiche bien le message **"Bienvenue dans l'application Dockerisée"** servi par le conteneur Nginx. Cela confirme que le Dockerfile est correct, que le mapping de port **8080:80** fonctionne, et que le fichier **index.html** a bien été copié dans l'image lors du build.

5.3 — Connexion à pgAdmin sur http://localhost:5050

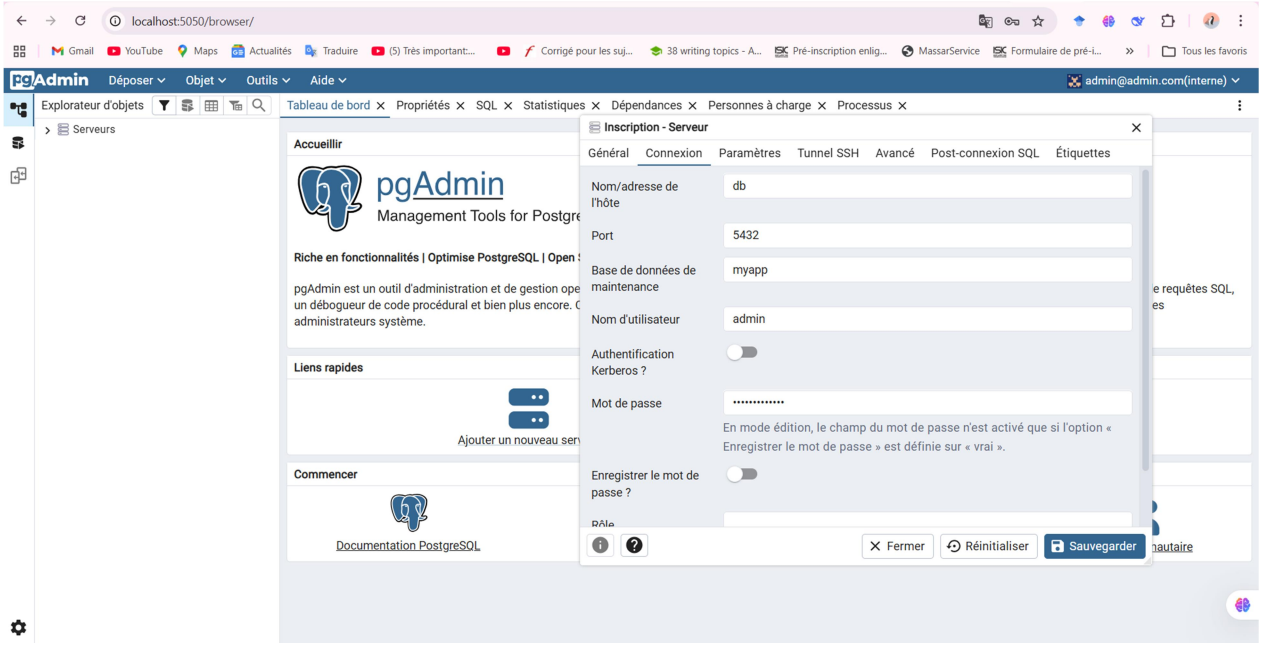

localhost:5050/login?next=/
Gmail
YouTube
Maps
Actualités
Traduire
(5) Très important:...
Corrigé pour les suj...
38 writing topics - A...
Pré-inscription enlig...
MassarService
Formulaire de pré-i...
Tous les favoris
pgAdmin
Se connecter
Adresse e-mail / Nom d'utilisateur
Mot de passe
Mot de passe oublié ?
Anglais
Se connecter

pgAdmin
Se connecter
admin@admin.com
•••••
Mot de passe oublié ?
Anglais
Se connecter

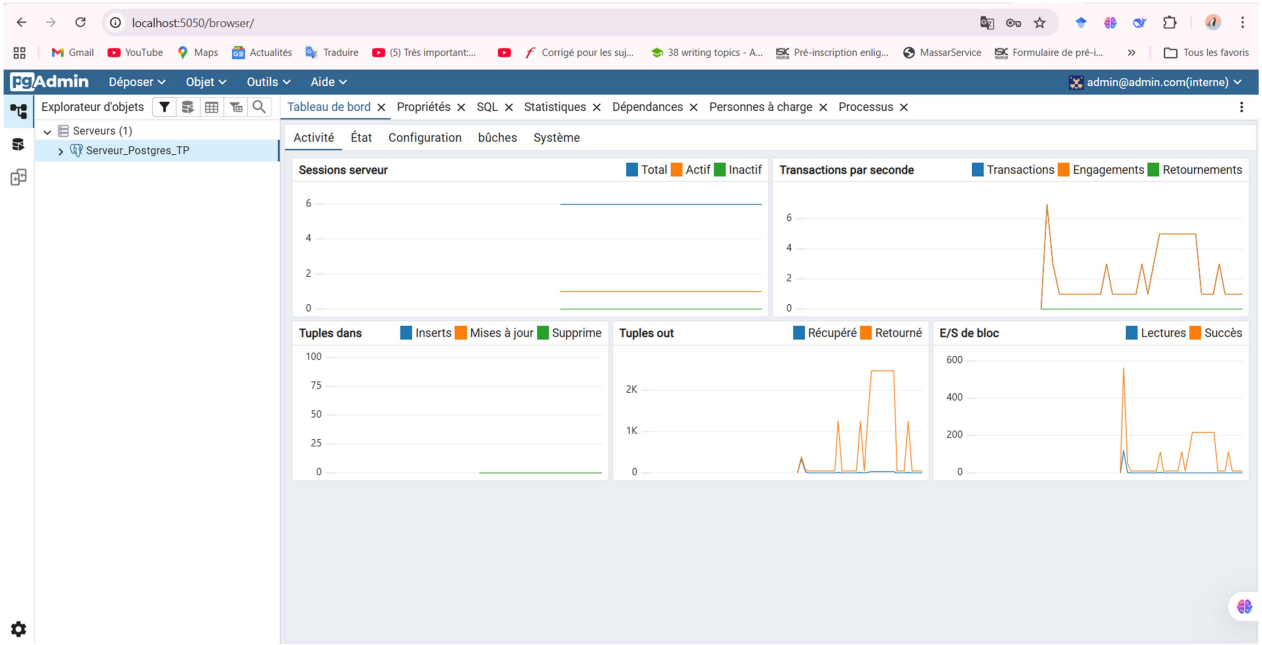


Interprétation : L'interface pgAdmin est accessible via le port 5050. On se connecte avec les identifiants **admin@admin.com / admin** définis dans les variables d'environnement du service pgAdmin dans le docker-compose.yml. Cette interface graphique facilite l'administration de la base de données PostgreSQL sans avoir à utiliser la ligne de commande.

5.4 — Ajout d'une connexion au serveur PostgreSQL dans pgAdmin



Interprétation : Dans pgAdmin, on ajoute un nouveau serveur avec les paramètres suivants : **Host: db** (nom du service Docker, résolvable grâce au réseau **app-network**), **Port: 5432** (port PostgreSQL par défaut), **Username: admin** et **Password: adminpassword**. Le nom d'hôte **"db"** fonctionne grâce au DNS interne de Docker Compose : chaque service est accessible par son nom dans le réseau commun.



5.5 — Persistance des données avec volume et redémarrage

Interprétation : Le volume nommé **db-data** est défini dans le docker-compose.yml et monté dans **/var/lib/postgresql/data** à l'intérieur du conteneur PostgreSQL. Après un **docker-compose down** suivi d'un **docker-compose up**, les données de la base sont intégralement restaurées car stockées dans le volume persistant géré par Docker. C'est un mécanisme essentiel en production : les données survivent aux cycles de vie des conteneurs.